

LLM Self-Supervised Fine-Tuning

Technical Project Report

Full-parameter continued pretraining of Qwen2.5-0.5B with controlled QA, retrieval, grounding and deployment evaluation

Validated runtime	Configuration
Compute	NVIDIA A30 24 GB SLURM GPU execution
Software	Python 3.11.11 PyTorch / Transformers FastAPI / Uvicorn
Base model	Qwen/Qwen2.5-0.5B
Adaptation	Full-parameter causal-LM continued pretraining; no LoRA/QLoRA
Corpus	SQuAD v2 context-only text 14,639 train contexts 512-token packing
Frozen downstream test	5,940 SQuAD v2 QA examples
Serving	Hybrid BM25 + E5 + RRF retrieval grounding/NLI monitoring dashboard
Testing	38 passing tests 78% backend statement coverage

Scope note: this is a hands-on end-to-end portfolio project designed to demonstrate production-oriented ML/LLM engineering practices. It is not presented as long-term commercial production experience.

September 2026

1. Executive summary

This project studies a deliberately narrow question: what changes when a small base causal language model is adapted with **self-supervised next-token training** on domain/task-adjacent text, without exposing downstream QA labels? The experiment uses SQuAD v2 context text as a context-only corpus, performs full-parameter continued pretraining of Qwen2.5-0.5B, and then evaluates the resulting model at several layers: causal-LM adaptation, frozen downstream QA, retrieval, retrieval-conditioned QA, evidence consistency, and live serving behavior.

The core result is intentionally non-monotonic. Continued pretraining improves validation language modeling - loss falls from 2.7491 to 2.6721 and perplexity from 15.6289 to 14.4701 - but downstream QA becomes worse. At the same time, when responses are evaluated against retrieved evidence, the adapted model is more often entailed by that evidence and less often contradictory. This demonstrates why **language-model fit, downstream task correctness, retrieval quality, and grounding are separate dimensions** and should not be collapsed into one score.

What was achieved

- Strict context separation between self-supervised train/validation corpora and the frozen downstream QA test set.
- Explicit full-parameter PyTorch/Hugging Face training loop with Optuna search and validation-loss checkpoint selection.
- Base-versus-adapted evaluation at causal-LM, QA, retrieval-conditioned QA, and grounding levels.
- BM25, dense E5, and hybrid Reciprocal Rank Fusion retrieval benchmark over 20,233 unique contexts.
- FastAPI serving with typed endpoints, live inference metrics, GPU telemetry, Swagger/OpenAPI, and a browser monitoring dashboard.
- 38 automated tests and 78% backend statement coverage, complemented by live CUDA API/dashboard smoke tests.

Interpretive boundary: NLI non-entailment is used only as an automatic hallucination proxy. A response can be entailed by the retrieved context while still being irrelevant or incorrect for the downstream question.

2. Experimental design and data boundaries

The project reuses SQuAD v2 in two different roles while enforcing a strict leakage boundary. The self-supervised training side sees only context text. The downstream QA side remains frozen and is evaluated only after model selection. QA answers, answerability labels, and test targets are not part of the continued-pretraining objective.

Dataset item	Value
Unique self-supervised training contexts	14,639
Unique self-supervised validation contexts	603
Frozen downstream test examples	5,940
Unique frozen test contexts	601
Packed block size	512 tokens
Packed training blocks	4,655
Packed validation blocks	201
Train/validation/test context overlap	0

Self-supervised objective

Contexts are deduplicated, tokenized, concatenated into a continuous token stream, and packed into fixed 512-token causal-LM blocks. Each token is predicted from previous tokens. There is no question/answer prompt template and no assistant-only supervision during adaptation.

```
SQuAD context text
|
v
deduplicate + audit
|
v
tokenize -> concatenate -> 512-token packing
|
v
next-token causal-LM objective
```

This design deliberately tests whether unlabeled continued pretraining on task-adjacent text transfers to QA behavior, rather than assuming that better domain language modeling will automatically imply better downstream task performance.

3. System architecture

Training, evaluation, retrieval and serving architecture

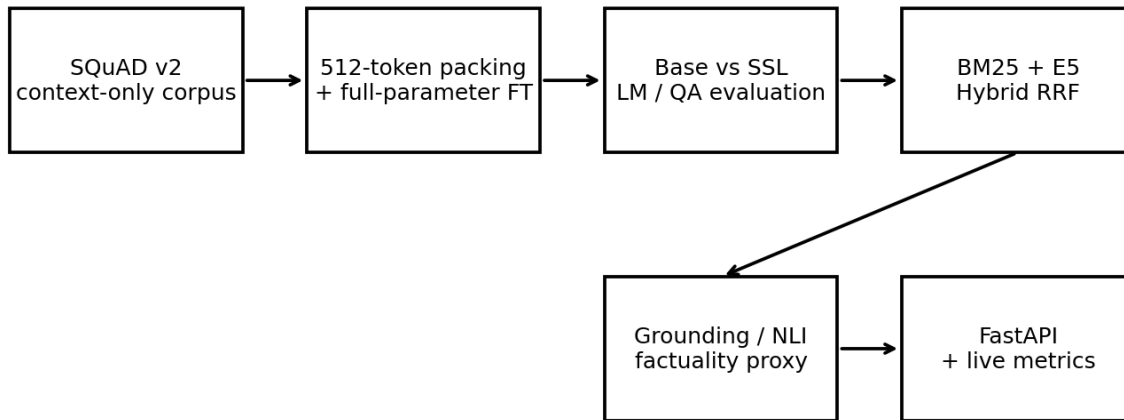


Figure 1. Training, evaluation, retrieval, grounding, and serving architecture.

Runtime request path

- The browser sends a question and model selection to FastAPI POST /answer.
- The service retrieves top evidence using BM25 + E5 with Reciprocal Rank Fusion (RRF, k=60).
- The selected Qwen base or self-supervised model generates deterministically from the retrieved context.
- A RoBERTa-large MNLI classifier estimates entailment, neutral, and contradiction probabilities for answer versus evidence.
- The response contains the answer, retrieved document, grounding signal, hallucination proxy, and component-level latency trace.
- GET /metrics exposes live request/error counts, latency summaries, GPU memory/utilization, and history; GET /benchmark exposes frozen offline results.

The retrieval layer is a controlled evaluation and deployment-grounding layer around the continued-pretraining experiment. It is not positioned as a replacement for a dedicated RAG project.

4. Full-parameter self-supervised training

The complete Qwen2.5-0.5B parameter set is optimized. No PEFT adapters are used and the project does not rely on Hugging Face Trainer. The custom loop keeps optimizer behavior, gradient accumulation, clipping, scheduler stepping, validation, and checkpoint selection explicit.

Training parameter	Final selected value
Model	Qwen/Qwen2.5-0.5B
Epochs	3
Per-device batch size	8
Gradient accumulation	2
Effective batch size	16
Precision	bf16
Learning rate	1.5207292892805172e-05
Weight decay	0.0407392908424372
Warmup ratio	0.031301953382248254
Scheduler	cosine
Max grad norm	1.9869442371619883
Seed	42

Hyperparameter selection

A broad Optuna search was followed by a refined search over a narrower promising region. The final full run uses refined trial 19 after an independent confirmation step. The selected checkpoint is the epoch with minimum self-supervised validation loss rather than the last epoch.

Selected refined trial 19: lr 1.5207292892805172e-05; weight decay 0.0407392908424372; warmup ratio 0.031301953382248254; cosine schedule; max grad norm 1.9869442371619883.

5. Training telemetry and checkpoint selection



Figure 2. Validation loss is lowest after epoch 1 and then increases slightly, supporting epoch-1 checkpoint selection.

Epoch	Validation loss	Interpretation
1	2.672082	Best checkpoint
2	2.675994	Slight degradation
3	2.677708	Further slight degradation

The full refined training job completed three epochs, but validation loss was best after epoch 1. The epoch-1 artifact at `results/checkpoints/self_supervised_v3_refined/best_model` is therefore the selected model. This is a small but consistent overfitting signal: continuing optimization reduces the fit to the training objective without improving held-out causal-LM validation.

The selection rule is fixed before downstream test analysis: downstream QA, retrieval-conditioned QA, and grounding metrics are diagnostic evaluations, not criteria for retrospectively choosing a different checkpoint.

6. Causal-LM adaptation result

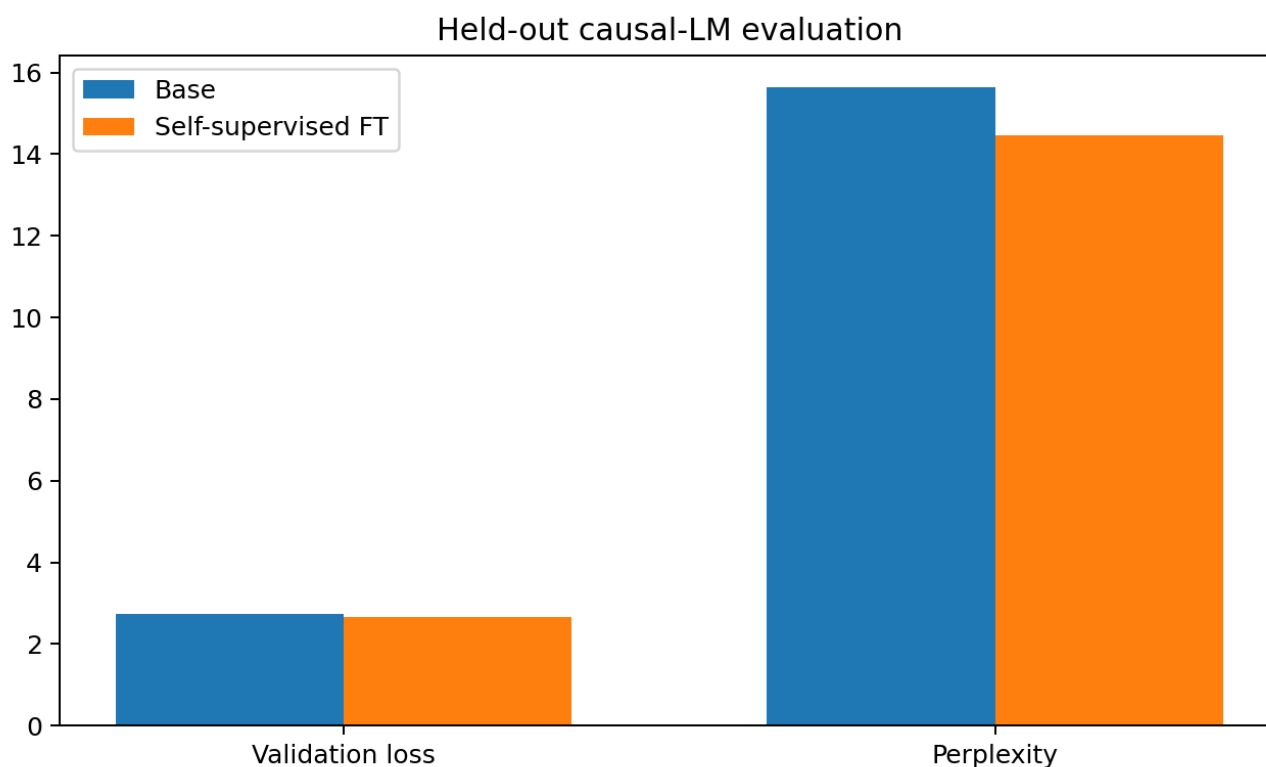


Figure 3. Base versus self-supervised validation loss and perplexity on the held-out context-only corpus.

Metric	Base model	Self-supervised	Relative change
Validation loss	2.749122	2.672082	-2.80%
Perplexity	15.6289	14.4701	-7.41%

On the objective it was trained for, the adapted model improves. This establishes that the continued-pretraining run is not simply a failed optimization. The next question is whether this language-model improvement transfers to the frozen QA task.

Key result: causal-LM improvement is real, but downstream task transfer must be measured independently rather than inferred from perplexity.

7. Frozen downstream QA: objective/task mismatch

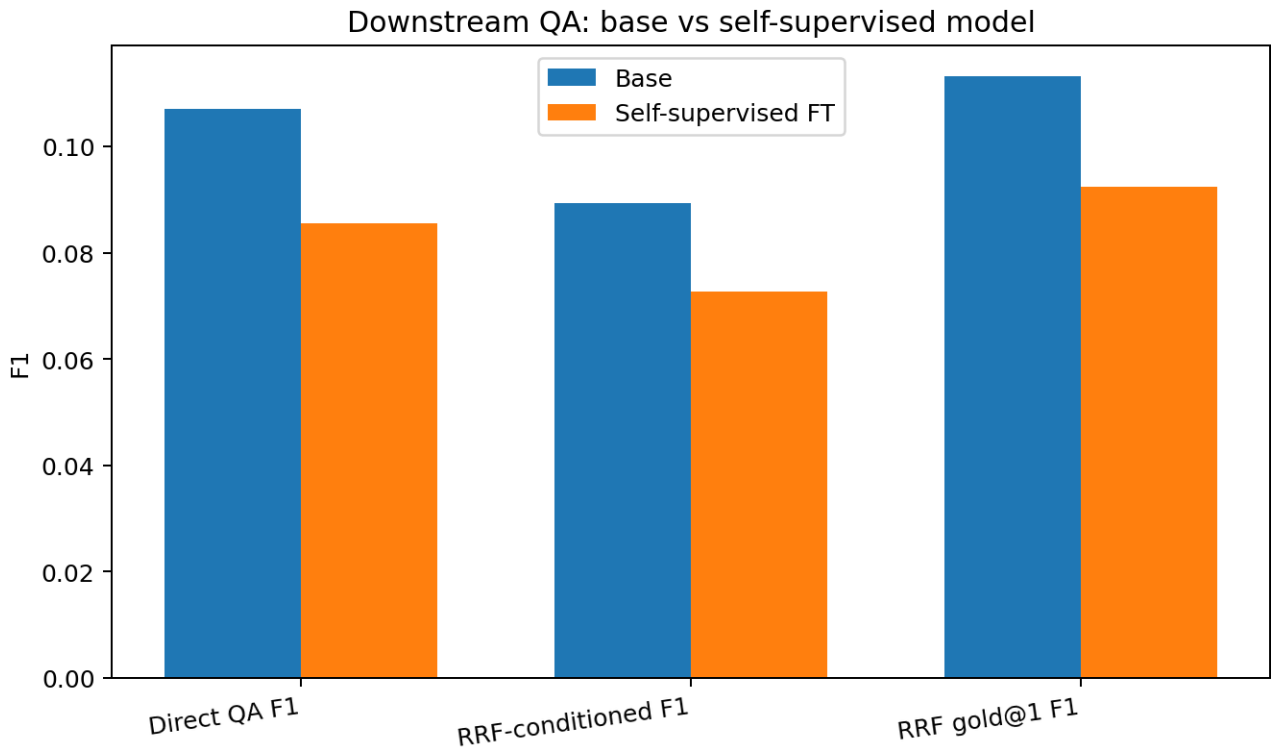


Figure 4. Frozen SQuAD v2 QA evaluation: the self-supervised model performs worse than the base model despite better LM validation.

Metric	Base	Self-supervised
Task F1	0.1070	0.0855
Task Exact Match	0.0042	0.0019
Answerable F1	0.2136	0.1707
Unanswerable abstention accuracy	0.0000	0.0000

Both models effectively fail to learn the task-specific abstention behavior because the self-supervised objective never teaches an explicit answer/abstain decision. More importantly, the adapted model also loses answerable QA performance. This is direct evidence of **objective/task alignment mismatch**: better modeling of context text does not imply better extraction of task answers.

For comparison, a separate supervised fine-tuning project using the same Qwen2.5-0.5B family and SQuAD v2 task reached task F1 0.7580 and EM 0.7271. That result is not reused as training evidence here; it simply illustrates how strongly task-aligned supervision changes behavior compared with context-only continued pretraining.

8. Retrieval benchmark

The retrieval corpus contains 20,233 unique official SQuAD train+validation contexts. The frozen 5,940 QA examples serve as retrieval queries; each example has an associated gold context. BM25 and dense E5 retrieval are evaluated independently, then fused with Reciprocal Rank Fusion over the top-10 candidate union.

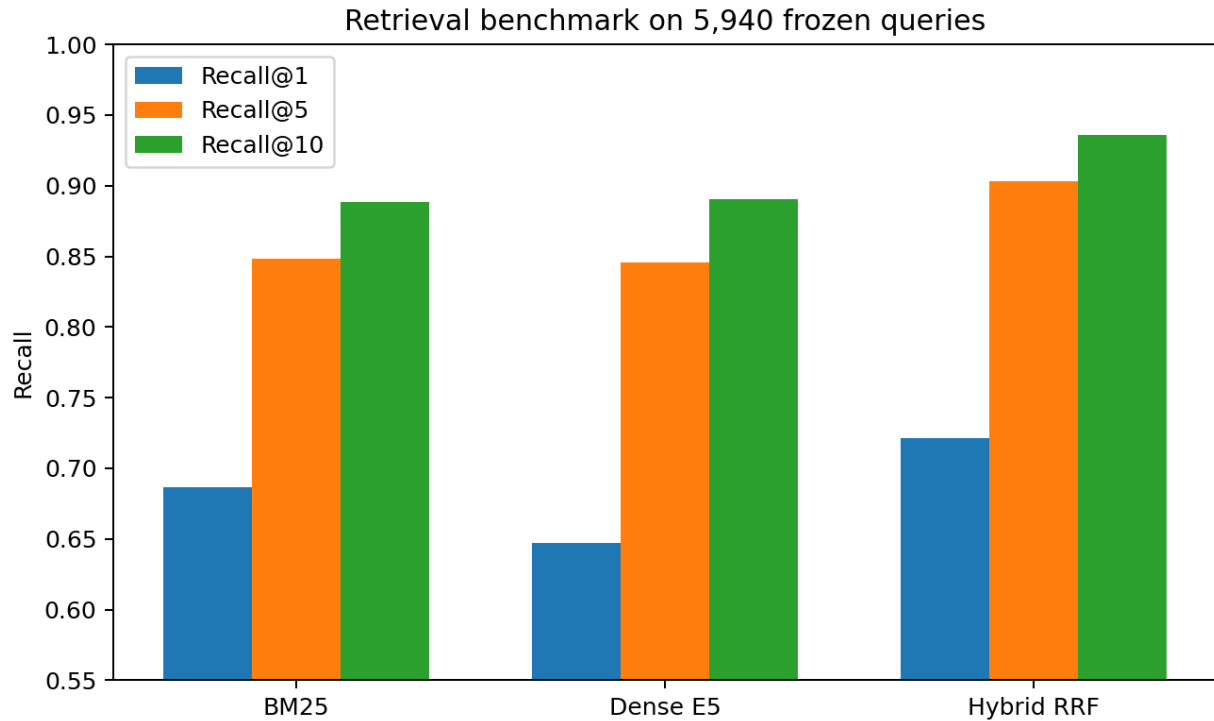


Figure 5. Retrieval recall and MRR comparison. Hybrid RRF is strongest across the controlled benchmark.

Method	R@1	R@3	R@5	R@10	MRR / MRR@10
BM25	0.6869	0.8121	0.8481	0.8886	0.7604
Dense E5-small-v2	0.6470	0.8024	0.8458	0.8904	0.7370
Hybrid RRF	0.7210	0.8620	0.9029	0.9357	0.7985

Hybrid RRF provides the strongest top-k retrieval performance. It is therefore used for retrieval-conditioned QA, grounding evaluation, and the deployment service. The RRF parameter is $k=60$.

9. Retrieval-conditioned QA

To separate retrieval failure from model/task failure, the existing downstream QA protocol is rerun with only one change: the context supplied to the model is the hybrid-RRF top-1 retrieved document. This preserves the generation prompt/protocol while introducing realistic retrieval noise.

Metric	Base + RRF	Self-supervised + RRF
Overall task F1	0.0894	0.0727
Task Exact Match	0.0035	0.0017
Answerable F1	0.1784	0.1451
F1 when gold context is rank 1	0.1133	0.0924
F1 when gold context is not rank 1	0.0275	0.0216
Unanswerable abstention accuracy	0.0000	0.0000

Hybrid retrieval places the gold context at rank 1 for 4,283 of 5,940 queries ($\text{Recall@1} = 0.7210$). QA quality is substantially higher when this succeeds, confirming that retrieval is one bottleneck. However, the self-supervised model remains below the base model even within the gold-at-1 subset, showing that retrieval success does not eliminate the separate model/task-alignment bottleneck.

Interpretation: there are at least two independently measurable failure sources - retrieving the right evidence and converting relevant evidence into the task-correct answer.

10. Grounding, evidence consistency, and factuality proxy

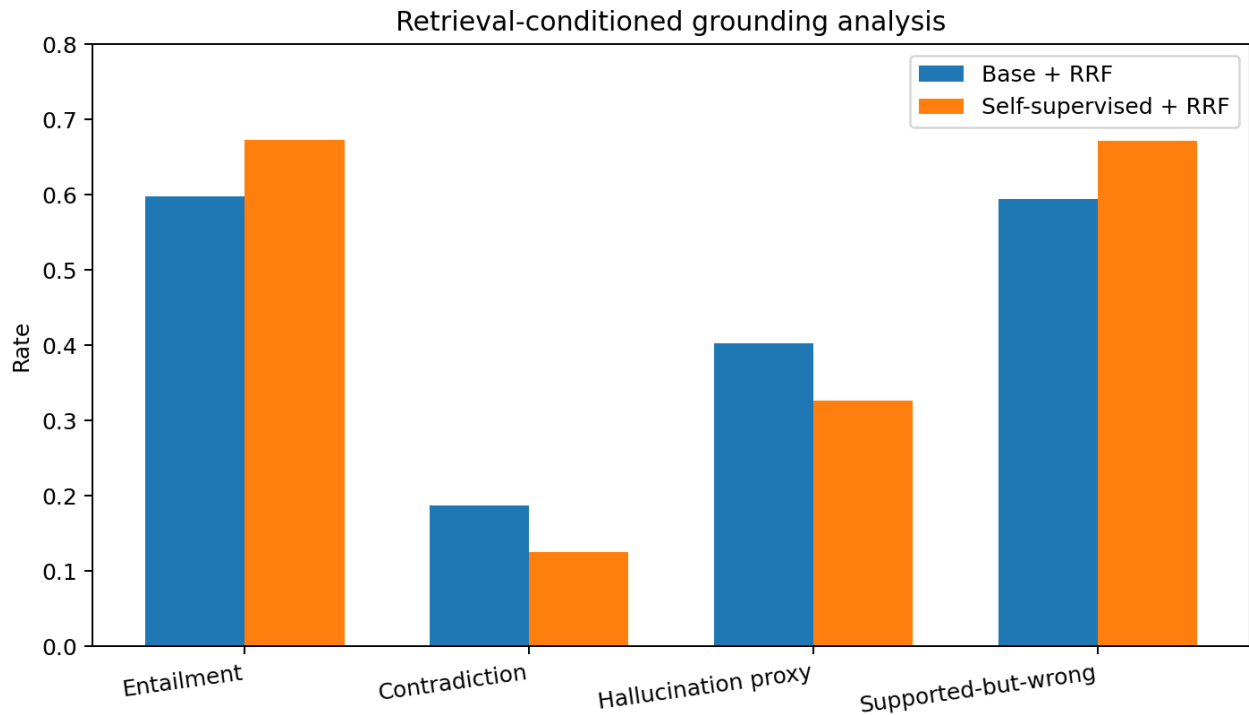


Figure 6. Grounding metrics with hybrid-RRF top-1 evidence. The adapted model is more often entailed and less often contradictory.

Metric	Base + RRF	Self-supervised + RRF
NLI entailment rate	0.5976	0.6729
NLI neutral rate	0.2148	0.2013
NLI contradiction rate	0.1875	0.1258
Automatic hallucination proxy	0.4024	0.3271
Supported but exact-match wrong	0.5948	0.6717
Supported with positive F1	0.2614	0.2577

With retrieved evidence, the adapted model gains about 7.53 percentage points in NLI entailment and reduces contradiction by about 6.18 points. Yet supported-but-exact-match-wrong responses increase. The result is important: **evidence consistency is not the same as answering the question correctly**. A model can produce statements supported by the context but irrelevant to the asked question.

Proxy definition: the report uses NLI non-entailment (neutral or contradiction) as an automatic hallucination proxy. This is a diagnostic signal, not factual ground truth and not a substitute for human or task-specific correctness evaluation.

11. FastAPI service and OpenAPI contract

The deployment backend loads the base model, adapted model, E5 retriever, persisted dense index, and NLI classifier once at application startup. The service requires CUDA in the canonical deployment configuration. Typed Pydantic schemas define system, retrieval, answer, grounding, trace, metrics, and benchmark responses.

Endpoint	Purpose
GET /health	Overall/component health and CUDA status
GET /ready	Model, retriever, and NLI readiness
GET /model-info	Model/retriever metadata and precision
POST /retrieve	BM25 + E5 + RRF retrieval with ranks/scores
POST /answer	Retrieve -> generate -> ground with latency trace
GET /metrics	Live inference/GPU observability
GET /benchmark	Frozen validated offline benchmark

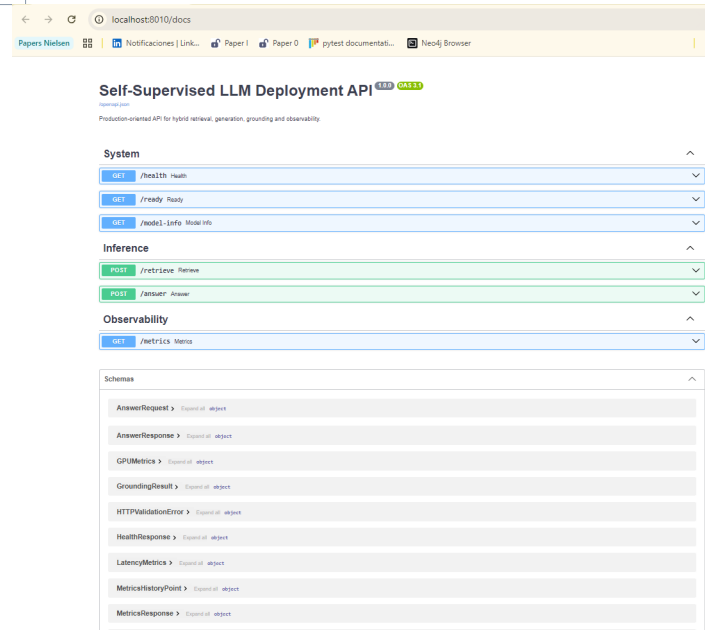


Figure 7. Swagger/OpenAPI overview of the deployed API surface.

12. Live retrieval and grounding behavior

A live retrieval request for “Who developed the theory of relativity?” returns HTTP 200 with three RRF-ranked documents. The first retrieved context discusses Einstein and general relativity, demonstrating the end-to-end hybrid retrieval path.

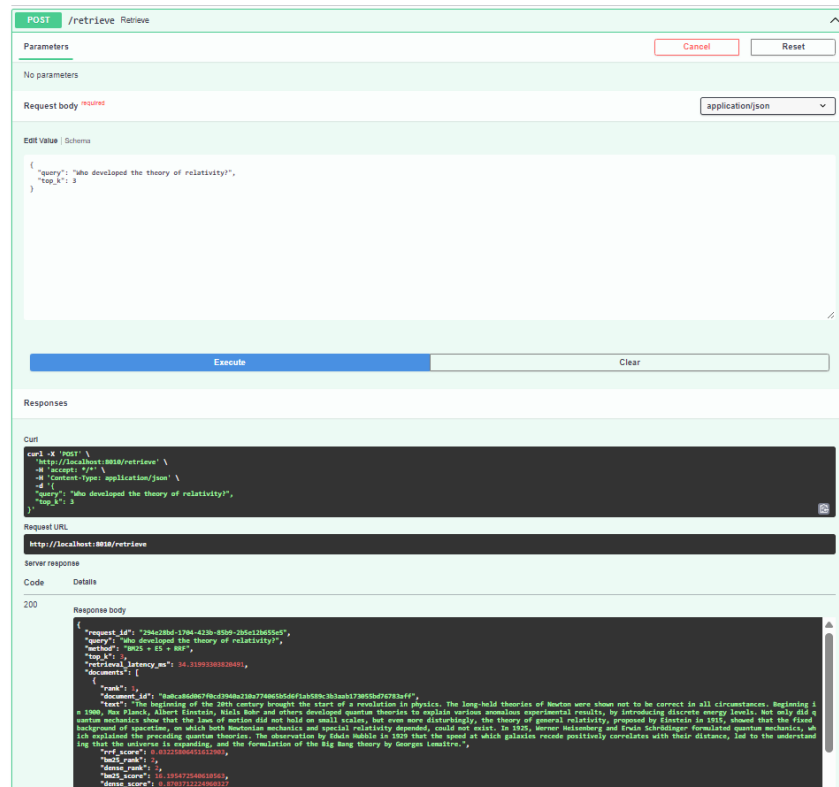


Figure 8. Live POST /retrieve response with BM25 rank, dense rank, component scores, and fused RRF score.

The answer endpoint intentionally exposes grounding diagnostics rather than hiding failure. In one live request, the generator produced a context-inconsistent refusal-style answer despite retrieved evidence about Einstein. The NLI layer classified the output as contradiction and set hallucination_proxy=true.

13. Failure detection as an observable runtime signal

The screenshot shows a web interface for testing API requests. At the top, there's a text area for the request body, which contains a JSON object: `{ "question": "Who developed the theory of relativity?", "model": "self_supervised" }`. Below this is an "Execute" button. The "Responses" section shows the result of a curl command: `curl -X 'POST' \ 'http://localhost:8010/answer' \ -H 'accept: */*' \ -H 'Content-Type: application/json' \ -d '{ "question": "Who developed the theory of relativity?", "model": "self_supervised" }'` . The "Request URL" is `http://localhost:8010/answer`. The "Server response" is a 200 status code. The "Response body" is a large JSON object: `{ "request_id": "7308c24f-c883-4ade-91a1-cdeb8dd3730a", "timestamp": "2026-09-30T04:42:37.156934Z", "question": "Who developed the theory of relativity?", "model": "self_supervised", "answer": "The assistant is not able to provide the answer to the question 'Who developed the theory of relativity?' as it is not mentioned in the provided context.", "retrieved_document": { "rank": 1, "document_id": "8a6c8d8d67f6cd3946a218a7740c5b5d6f1ab58c3b3ab173055bd76783e6f", "text": "The beginning of the 20th century brought the start of a revolution in physics. The long-held theories of Newton were shown not to be correct in all circumstances. Beginning in 1900, Max Planck, Albert Einstein, Niels Bohr and others developed quantum theories to explain various anomalous experimental results, by introducing discrete energy levels. Not only did quantum mechanics show that the laws of nature did not hold on small scales, but even more disturbingly, the theory of general relativity, proposed by Einstein in 1915, showed that the fixed background of spacetime, on which both Newtonian mechanics and special relativity depended, could not exist. In 1923, Werner Heisenberg and Erwin Schrödinger formulated quantum mechanics, which explained the preceding quantum theories. The observation by Edwin Hubble in 1929 that the speed at which galaxies recede positively correlates with their distance, led to the understanding that the universe is expanding, and the formulation of the Big Bang theory by Georges Lemaitre." }, "rrf_score": 0.8322506451612983, "lmr_score": 0.835472340638563, "dense_rank": 1, "dense_score": 0.156263417005539, "grounding": { "lexical_support": false, "all_label": "contradiction", "entailment_probability": 0.805339354440957308, "neutral_probability": 0.156263417005539, "contradiction_probability": 0.839043319233113, "hallucination_proxy": true } }, "Response headers": { "content-length": 2023, "content-type": "application/json", "date": "Wed, 30 Sep 2026 04:42:35 GMT", "server": "uvicorn" }`

Figure 9. Live POST /answer example where the grounding layer flags contradiction and the automatic hallucination proxy.

This failure is useful evidence for the project because it shows that the observability layer is not merely decorative. The API returns the retrieved evidence, generated answer, NLI probabilities, proxy flag, and component trace, allowing a client or operator to distinguish a fluent response from one that is poorly supported by the selected evidence.

The system does not automatically suppress or rewrite such outputs in this portfolio version. The focus is on making failure visible and measurable; a stricter production policy could gate answers based on grounding thresholds or explicit abstention logic.

14. Live observability and GPU telemetry

GET /metrics maintains a rolling inference window with request/error counts, requests per minute, retrieval/generation/grounding/end-to-end latency summaries, GPU memory, GPU utilization, and history points for dashboard plotting. Health/metrics polling is separated from inference counters so the displayed request count reflects inference activity.

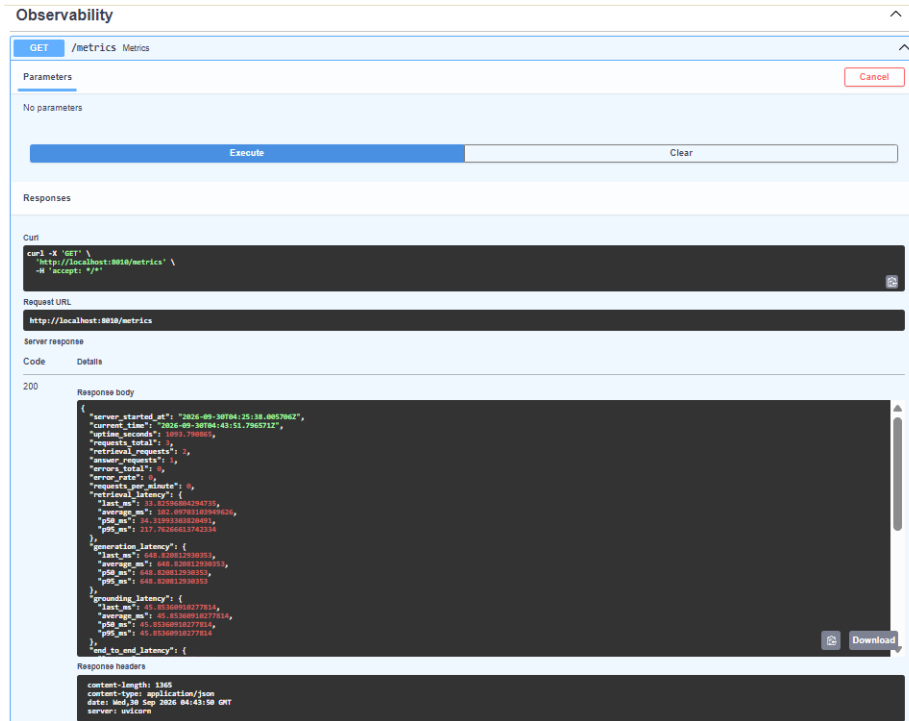


Figure 10. Swagger view of live latency and observability metrics.

Validated live metrics snapshot

```
{
  "requests_total": 3,
  "errors_total": 0,
  "error_rate": 0.0,
  "retrieval_latency": {"average_ms": 102.10, "p95_ms": 217.76},
  "generation_latency_ms": 648.82,
  "grounding_latency_ms": 45.85,
  "end_to_end_latency_ms": 740.50,
  "gpu": {
    "device_name": "NVIDIA A30",
    "memory_allocated_gib": 3.34,
    "memory_total_gib": 23.60,
    "memory_usage_percent": 14.17
  }
}
```

This snapshot is a small live smoke-session observation, not a throughput/load-test claim.

15. Browser monitoring dashboard

The static HTML/CSS/JavaScript dashboard is designed to make the production-oriented aspects of the project immediately visible. It clearly separates **live runtime telemetry** from the **validated offline benchmark** so frozen experimental results are not presented as if they changed in real time.

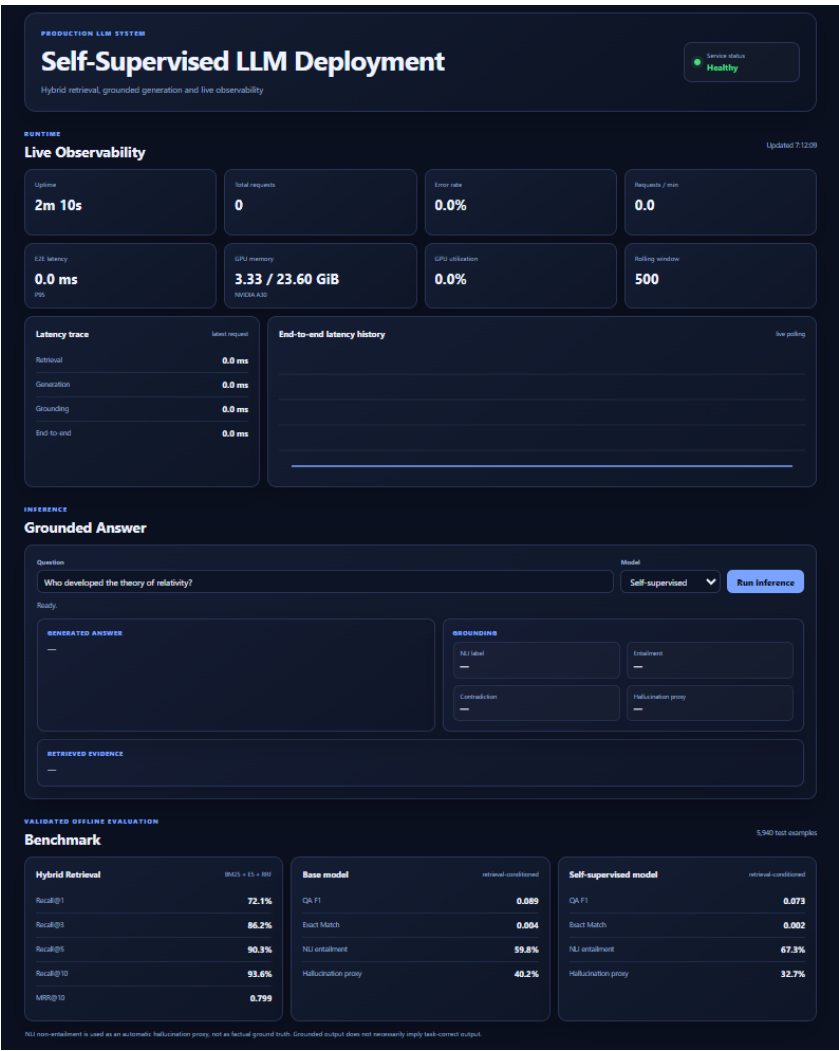


Figure 11. Dashboard overview before inference: health, GPU state, latency panels, inference controls, and frozen benchmark summary.

Live cards show health, uptime, inference requests, error rate, requests/minute, P95 end-to-end latency, GPU memory/utilization, and rolling-window size. The history panel refreshes from /metrics. The lower benchmark section exposes hybrid retrieval recall/MRR and base-versus-adapted QA/grounding metrics from the frozen 5,940-example test set.

16. Dashboard inference: detected grounding failure

The dashboard updates the same live observability panels used by the API while displaying the request-level evidence and grounding result. In this negative example, the output is classified as contradiction and the hallucination proxy is visibly flagged.

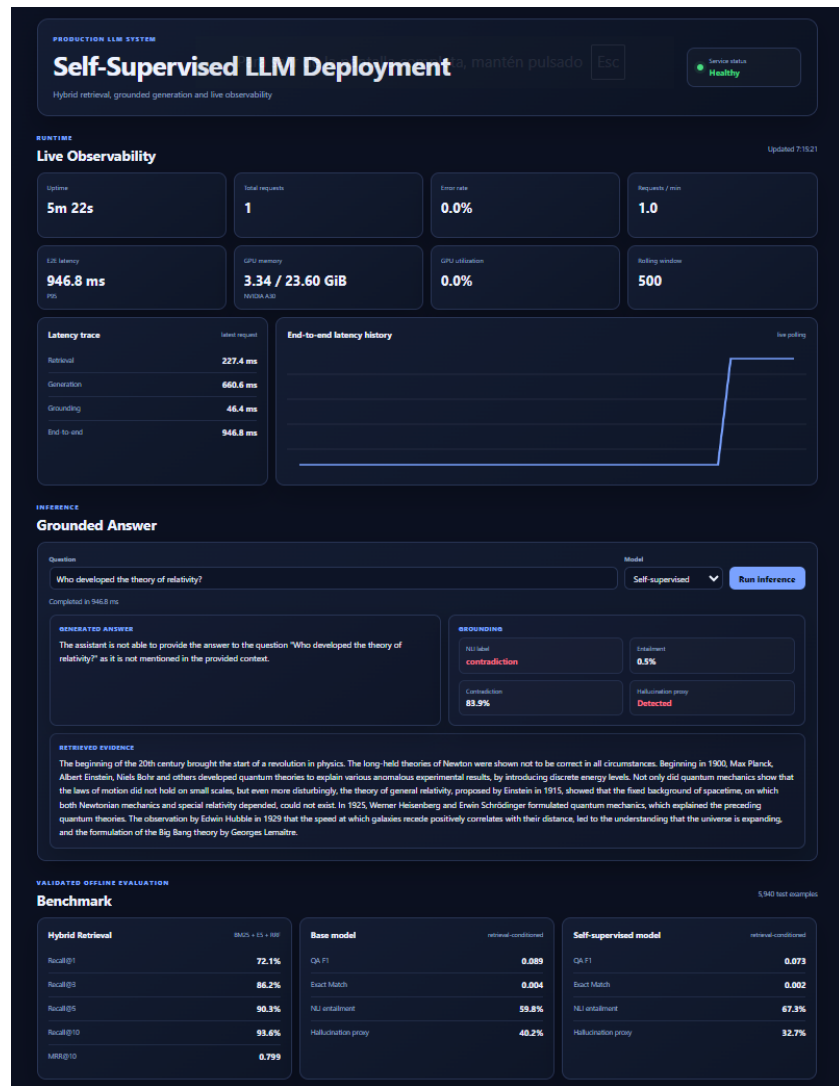


Figure 12. Negative case: contradiction is highlighted and the hallucination proxy is detected while live latency/history update.

17. Dashboard inference: positive grounding case

A second live query asks which physicist proposed general relativity in 1915. The retrieved context contains the supporting sentence, the model answers Albert Einstein, NLI entailment reaches 95.9%, contradiction is 1.7%, and the proxy is not detected.

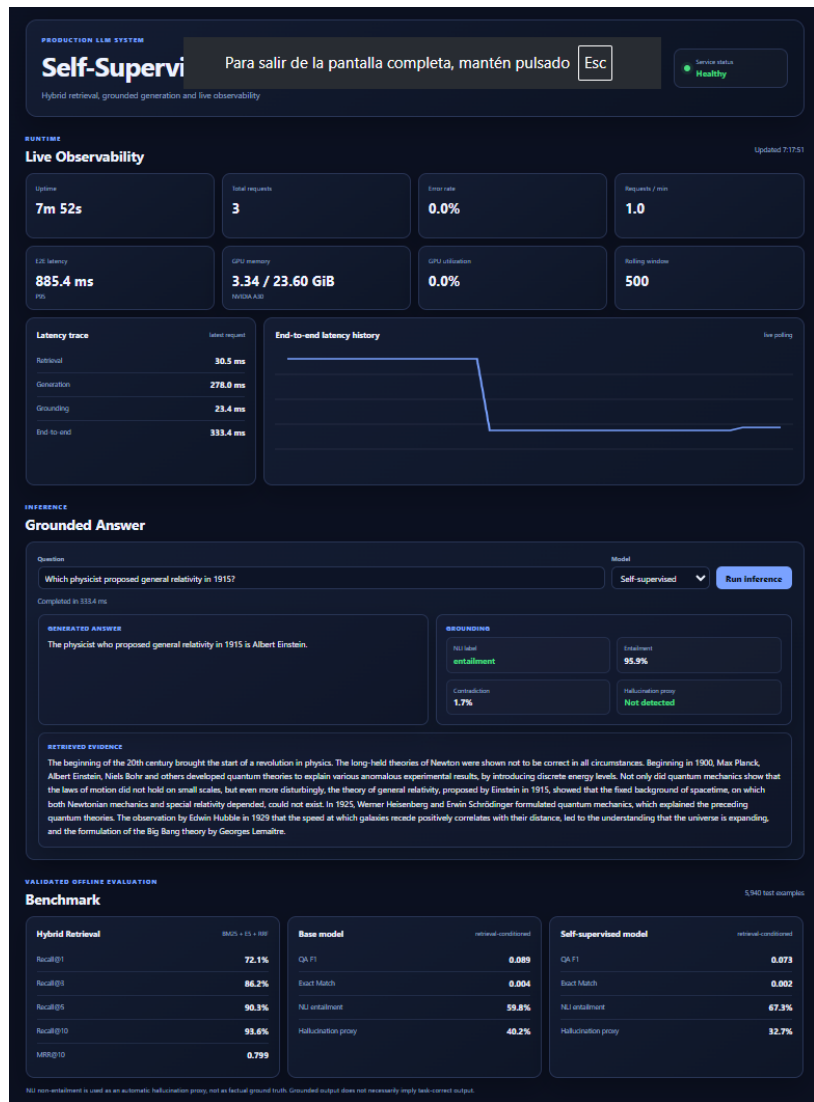


Figure 13. Positive case: “Albert Einstein” is supported by the retrieved context; NLI entailment is 95.9% and the proxy is not detected.

18. Frozen benchmark endpoint, testing, and coverage

GET /benchmark exposes only previously validated evaluation artifacts. It does not recalculate benchmark scores on each live request. This preserves a clean distinction between offline experiment results and online service telemetry.

```
benchmark_name: RRF retrieval-conditioned QA and grounding benchmark
test_examples: 5940
retrieval: R@1 0.7210 | R@3 0.8620 | R@5 0.9029 | R@10 0.9357 | MRR@10 0.7985
base: task F1 0.0894 | NLI entailment 0.5976 | hallucination proxy 0.4024
self_supervised: task F1 0.0727 | NLI entailment 0.6729 | hallucination proxy 0.3271
note: NLI non-entailment is an automatic proxy, not factual ground truth.
```

Automated tests

Module	Coverage
backend/benchmark.py	100%
backend/main.py	92%
backend/schemas.py	100%
backend/service.py	66%
TOTAL backend	78%

38 passed in 9.03s

The remaining untested service code is concentrated in real model loading, dense-model execution, generation, NLI inference, and CUDA-specific branches. Those paths were exercised in live GPU API/dashboard smoke tests rather than being extensively mocked solely to maximize the coverage number.

Reproducibility boundary

Configuration, data construction, fixed seeds, training/evaluation scripts, SLURM wrappers, Optuna exports, deployment-index construction, locked dependencies, and tests are tracked. Fixed seeds do not imply bitwise-identical CUDA results across hardware/software environments.

19. Limitations and conclusions

Known limitations

- The self-supervised objective improves causal-LM validation loss/perplexity but does not improve downstream SQuAD QA accuracy in this experiment.
- Neither base nor adapted model learns reliable SQuAD v2 unanswerable-question abstention from context-only continued pretraining.
- NLI non-entailment is an automatic proxy and cannot establish factual correctness by itself.
- A statement can be supported by retrieved evidence yet still be irrelevant or wrong for the task question.
- Retrieval relevance is evaluated against the associated SQuAD context, a controlled benchmark target rather than a universal relevance judgment.
- The dense retriever is E5-small-v2; this is not an exhaustive retrieval-model study.
- The browser service is a portfolio deployment demonstration, not a 24/7 commercial service; no concurrency/stress, autoscaling, authentication, or CI/CD production claim is made.

Conclusions

The project provides a controlled example of why LLM systems should be evaluated at multiple layers. The adapted model is demonstrably better at the self-supervised objective, demonstrably worse at the frozen QA task, and demonstrably more consistent with retrieved evidence under the NLI proxy. None of these facts cancels the others.

From an engineering perspective, the work extends beyond training a checkpoint: it includes leakage-aware data construction, explicit optimization, hyperparameter selection, frozen evaluation, retrieval benchmarking, grounding diagnostics, typed serving, live telemetry, browser monitoring, GPU execution, and automated testing. The resulting repository is intended to be reproducible and inspectable rather than to hide limitations behind a single aggregate score.

Primary takeaway: better language modeling, better retrieval, better grounding, and better downstream task accuracy are related but distinct objectives. A production-oriented evaluation stack should measure each explicitly.

Repository deliverables

- `README.md` - reproducibility, usage, architecture, results, and limitations.
- `technical_report.pdf` - illustrated experimental and deployment evidence.
- `docs/screenshots/` - runtime evidence captured from Swagger and the monitoring dashboard.
- `tests/` - API, benchmark, schema, service, metrics, BM25/RRF, and helper tests.